PORTFOLIO • AI ENGINEER

Stéphane Gonzalez
AI Engineer

Je conçois des solutions d'intelligence artificielle de la donnée jusqu'au service en production : fine-tuning de LLM, RAG, agents, API, CI/CD, cloud et monitoring.

Machine Learning | Deep Learning | LLM & fine-tuning | RAG & agents | MLOps | Cloud Azure

---

11
Projets IA menés

6
Projets avec CI/CD

Azure
SortirIA en production

90,6 %
Triage médical : cas bien classés

PROFIL

À propos

De l'informatique et de la relation client à la conception de systèmes d'intelligence artificielle.

Après une longue expérience professionnelle dans l'informatique et l'électronique, j'ai choisi de me spécialiser dans l'intelligence artificielle avec la formation Ingénieur IA d'OpenClassrooms.

Mes projets m'ont conduit du Machine Learning classique au déploiement d'API, puis au RAG, aux agents et au fine-tuning de LLM, toujours avec la même exigence : un système testé, reproductible et mesuré.

Objectif professionnel : rejoindre une équipe comme AI Engineer pour concevoir et mettre en production des solutions à base de LLM, avec une attention particulière à l'évaluation et au monitoring.

RÉALISATIONS

Projets

Onze projets menés pendant ma formation, du plus récent au plus ancien. Chacun est livré avec son code ou ses livrables, et chaque carte y renvoie. Tous sont des projets individuels : chaque carte précise mon rôle et la principale difficulté rencontrée. Le dernier, SortirIA, est mon projet personnel : il réunit les briques des précédents dans un service en ligne.

Projet personnel • RAG • Agent • Production sur Azure

Oct.-nov. 2026 · en ligne

SortirIA

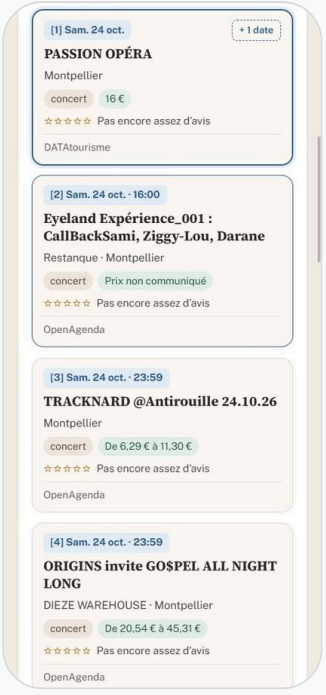
Un assistant de sorties pour Montpellier et le bassin de Thau : on écrit sa demande comme on la dirait (« un concert gratuit ce week-end à Sète »), SortirIA répond avec des sorties qui respectent la date, le lieu, le public et le budget. Chaque nuit, un agent LangGraph collecte et vérifie les agendas ouverts (OpenAgenda, DATAtourisme, programmes en PDF) ; la recherche combine filtres SQL, mots et vecteurs (pgvector) et Mistral rédige la réponse. Le service est en ligne sur Azure, déployé à chaque push et surveillé.

Mon rôle — Seul, de bout en bout : cadrage, demandes d'autorisation aux sources, code, déploiement sur Azure et suivi en production.

Le défi — Respecter à coup sûr la date, le lieu, le public et le budget d'une demande écrite librement, avec des agendas hétérogènes à collecter et vérifier chaque nuit.



L'accueil : on écrit sa demande comme on la dirait.



La réponse : des cartes datées, avec prix et source citée.

98,7 %
Contraintes respectées

97,8 %
Pertinence des 3 premières sorties

98,6 %

Accord du juge LLM avec l'humain

6,4 s

Temps de réponse P95

0,12 $

Coût Mistral pour 1 000 questions

CONDUITE DE PROJET

1. Besoin

Trouver une sortie qui convient vraiment (date, lieu, public, budget), sans parcourir dix agendas.

2. Audit des données

Sources ouvertes et autorisées seulement ; chaque fiche vérifiée, dédoublonnée, attribuée à sa source.

3. Solution

Agent LangGraph la nuit, PostgreSQL + pgvector, FastAPI, Angular, Mistral ; code repris de mes projets précédents et adapté.

4. Aide à la décision

Seuils fixés avant de mesurer (95 % de contraintes, 70 % de pertinence) ; juge calibré sur l'humain ; limites assumées.

5. Contrôle

Environ 10 $ par mois ; évaluation rejouée en CI (déploiement bloqué sous 95 %) ; Grafana, MLflow, suivi de la dérive, alertes et budget Azure.

THÈMES MLOPS DU PROJET FINAL

La consigne propose quatre thèmes. Voici ce que SortirIA en couvre, et comment, pour un service qui appelle Mistral au lieu d'entraîner son propre modèle.

Cycle de vie des modèles

Couvert

CI à chaque push : tests sur un vrai PostgreSQL et évaluation de la recherche rejouée, déploiement bloqué sous 95 %. CD vers Azure, contrôle du service en ligne toutes les 3 h, MLflow pour chaque collecte et chaque évaluation.

Suivi de la dérive Couvert	Chaque nuit, les 7 derniers jours comparés aux 28 précédents (PSI) : ce que les visiteurs demandent (zone, public, type de sortie), ce que SortirIA répond (exacte, élargie, introuvable ; latence, avis) et ce que les sources apportent (volumes, rejets, juge de qualité). Rapport le lundi, alerte si la dérive est forte, suivi dans Grafana et MLflow.
Réentraînement automatique Adapté	Pas de modèle à réentraîner : SortirIA appelle Mistral. La boucle d'amélioration est l'agent du lundi : il lit les recherches ratées et les pouces bas, puis ouvre une pull request par correction (synonyme, agenda à lire). Elle est testée en CI, je la valide, puis elle est déployée. Un fine-tuning est prévu quand les avis réels suffiront.
Versionnage des données et des modèles Partiel	Chaque réponse garde la version de sa consigne, et MLflow celle du juge (v7) et de la lecture des PDF. Les jeux d'évaluation et la configuration des sources sont versionnés dans Git. Pas de DVC ni de Model Registry : aucun modèle entraîné à enregistrer.

LangGraph · Mistral · PostgreSQL / pgvector · FastAPI · Angular · Azure Container Apps · GitHub Actions · Grafana · MLflow · Airflow

Site en ligne → **Code →** **Rapport de conduite de projet →** **Rapport en PDF →**

Dossier du projet → **Documentation →**

LLM • Fine-tuning • Production

Sept.-oct. 2026

Agent IA de triage médical

Un petit modèle de langage (Qwen3-1.7B) spécialisé pour classer un patient des urgences en P1, P2 ou P3 et justifier sa décision, en français et en anglais. Entraîné par SFT puis DPO avec QLoRA sur une carte graphique de 6 Go, servi par vLLM derrière une API FastAPI journalisée et surveillée.

Mon rôle	Seul : audit et anonymisation des données, entraînement, service du modèle, tests et documentation.
Le défi	Spécialiser un modèle sur une carte de 6 Go sans sous-trier les urgences : le premier entraînement (Qwen3-Base) a échoué, je l'ai diagnostiqué puis corrigé.

Chaque brique est marquée opérationnelle ou partielle : l'état réel, pas l'état visé.

453 / 500	90,6 %	93,4 %
Cas du test gelé bien classés	F1 macro	Urgences P1 reconnues

5,4 %	1,13 s
Sous-triage	Latence P95

CONDUITE DE PROJET

1. Besoin

Aider les infirmiers d'accueil à trier les patients, avec une explication.

2. Audit des données

164 477 lignes publiques auditées, anonymisées (Presidio), réduites à 5 000 cas équilibrés.

3. Solution

QLoRA, SFT puis DPO, modèle fusionné et servi par vLLM.

4. Aide à la décision

Grille go / no-go et pilote en quatre phases, le modèle en second avis.

5. Contrôle

4 semaines, 6 Go de GPU, CI avec 40 tests, test gelé rejoué sur le modèle servi.

Qwen3 · PyTorch · QLoRA · SFT / DPO · vLLM · FastAPI · Prometheus / Grafana · W&B · GitHub Actions

Code → **Documentation →** **Rapport technique →**

Agent IA • RAG

Août 2026

Chess Agent

Aide à l'apprentissage des ouvertures d'échecs. Un agent LangGraph combine Stockfish, Lichess, un RAG sur Wikichess et un LLM local pour expliquer une position en français.

Mon rôle	Seul : architecture de l'agent, backend FastAPI, interface Angular, tests et CI.
Le défi	Ne pas laisser le LLM inventer un coup ou une évaluation : Stockfish calcule, les sources apportent le contexte, le LLM explique seulement.

1 029
Tests automatisés

≥ 86 %
Couverture exigée par la CI

LangGraph · Milvus · Ollama · FastAPI · MongoDB · Angular

Code → **Documentation →**

Data engineering

Juillet 2026

CheckIt.AI

Moteur de collecte multi-sources réutilisable : un moteur par type de source, un adaptateur et une configuration YAML par fournisseur. Validation, déduplication et contrôle qualité, orchestrés par Airflow. Premier cas d'usage : le fact-checking. SortirIA le réutilise.

Mon rôle	Seul : architecture du moteur, connecteurs, orchestration Airflow et tableau de bord.
Le défi	Faire entrer des sources très différentes dans un même format fiable (dates, liens, doublons, images vérifiés) sans dupliquer le code.

21

Sources branchées (RSS, API, web, réseaux sociaux, datasets)

Airflow · PostgreSQL · YAML · Docker · Streamlit

Code → **Documentation →**

Reinforcement Learning

Juillet 2026

Atterrissage LunarLander

Entraînement d'agents DQN et PPO pour poser un module lunaire, optimisation des hyperparamètres et restitution dans une API et un tableau de bord.

Mon rôle	Seul : entraînement des agents, réglage des hyperparamètres, API et tableau de bord.
Le défi	Obtenir un atterrissage stable et reproductible : comparer DQN et PPO, puis régler leurs hyperparamètres avec Optuna.

Un atterrissage réussi par l'agent entraîné.

Stable-Baselines3 · Gymnasium · Optuna · FastAPI

Code →

Vision • Semi-supervisé

Juin 2026

CurelyticsIA · BrainScanAI

Détection de tumeurs sur des IRM cérébrales avec peu d'images annotées : caractéristiques extraites par ResNet50, ACP et clustering, pseudo-étiquettes, puis entraînement d'un ResNet18. Étude de coût pour un budget de 5 000 €.

Mon rôle	Seul : pipeline semi-supervisé, évaluation et étude de coût.
Le défi	Apprendre avec très peu d'images annotées sans manquer de tumeur : le rappel prime sur tout le reste.

Supervisé seul, puis semi-supervisé avant et après fine-tuning.

1,00	0,98	0,95
Rappel (tumeurs détectées)	F2-score	Accuracy, contre 0,90 en supervisé seul

PyTorch · ResNet · Clustering · Pseudo-labels

Code →

Cadrage • Gestion de projet

Mai 2026

Cadrage du POC Fashion-Insta

Cadrage d'un système de recommandation de vêtements par la vision pour une enseigne de mode : besoin métier, choix de l'approche (CLIP + recherche vectorielle), planning de 3 semaines, budget de 13 à 15 k€, KPI de succès, architecture cible sur Azure, ROI et RGPD.

Mon rôle	Seul, dans le rôle du chef de projet IA : cadrage, chiffrage et présentation au client.
Le défi	Permettre une décision avant d'écrire du code : KPI, budget, planning et contraintes RGPD fixés pour un go / no-go.

Cadrage | Budget & ROI | Architecture Azure | RGPD | CLIP

Livrable : présentation de 52 diapositives

ML • MLOps

Avril-mai 2026

Scoring crédit Prêt à dépenser

Modèle de scoring mis en production : API, suivi des expériences MLflow, monitoring de la dérive avec Evidently, optimisation de l'inférence avec ONNX.

Mon rôle	Seul : API, suivi MLflow, monitoring de la dérive, optimisation et CI.
Le défi	Garder un modèle fiable après son déploiement : tracer chaque appel, détecter la dérive et accélérer l'inférence.

11,8 ms	15,7 ms	−18,6 %
Latence API moyenne	Latence API P95	Temps de traitement après optimisation

LightGBM | MLflow | Evidently | ONNX | GitHub Actions

Code →

LLM • RAG

Mars-avril 2026

Assistant RAG OpenAgenda

Recommandation d'événements culturels : préfiltrage structuré (ville, date, type), recherche sémantique FAISS, classement explicable et réponse générée par Mistral. Évaluation RAGAS sur des questions claires, ambiguës et hors corpus.

Mon rôle	Seul : collecte OpenAgenda, pipeline RAG, évaluation et conteneurisation.
Le défi	Comprendre une demande libre, parfois floue, et ne répondre qu'avec les événements retrouvés, sans en inventer.

LangChain · FAISS · Mistral · RAGAS · Docker

Code →

ML • Déploiement

Février 2026

TechNova Partners

Prédiction du départ des employés, déployée en API FastAPI sécurisée avec base PostgreSQL et traçabilité des prédictions. L'API est en ligne.

Mon rôle	Seul : base de données, API sécurisée, tests, CI/CD et mise en ligne.
Le défi	Rendre le modèle utilisable en production : aucune préparation des données dans l'API, chaque prédiction tracée en base.

scikit-learn · FastAPI · PostgreSQL · Hugging Face Spaces

Code → **API en ligne →**

Vision • Segmentation

Oct.-nov. 2025

ModeTrends

Segmentation de vêtements dans des photos d'influenceurs via un modèle Hugging Face appelé par API, puis analyse des tendances par catégorie de vêtement.

Mon rôle	Seul : appels au modèle, mesure de sa qualité et analyse des tendances.
Le défi	Juger un modèle que je n'ai pas entraîné : l'IoU et le Dice par vêtement montrent que les petits éléments (chaussures, bras) sont mal segmentés.

Qualité de la segmentation, vêtement par vêtement.

Hugging Face API · pandas · Matplotlib

Code →

AUTO-ÉVALUATION

Compétences

Chaque niveau est une auto-évaluation de mon autonomie, et chaque compétence renvoie aux projets où je l'ai mise en œuvre.

Notions · En progression · Opérationnel

Autonome

Machine Learning & données

Python — Autonome

Scoring crédit · Chess Agent · Triage médical · SortirIA

pandas / NumPy — Opérationnel

Scoring crédit · CurelyticsIA · CheckIt.AI

scikit-learn, LightGBM, XGBoost — Opérationnel

TechNova · Scoring crédit

SQL, PostgreSQL, pgvector — Opérationnel

TechNova · Scoring crédit · CheckIt.AI · SortirIA

Optuna, SHAP — Opérationnel

TechNova · Scoring crédit

Deep Learning & LLM

PyTorch — Opérationnel
CurelyticsIA · Triage médical

Hugging Face Transformers, PEFT — Opérationnel
Triage médical

Fine-tuning LoRA / QLoRA, SFT, DPO — Opérationnel
Triage médical

RAG : LangChain, FAISS, Milvus, pgvector, RAGAS — Opérationnel
RAG OpenAgenda · Chess Agent · SortirIA

Évaluation des LLM : juge LLM calibré sur l'humain, kappa, AC1 — Opérationnel
RAG OpenAgenda · SortirIA

Agents : LangGraph — Opérationnel
Chess Agent · SortirIA

Vision : CNN, OpenCV, segmentation — En progression
CurelyticsIA · ModeTrends

Reinforcement Learning : Stable-Baselines3 — En progression
LunarLander

Anonymisation : Presidio — En progression
Triage médical

TensorFlow — Notions
À acquérir

Développement & déploiement

FastAPI

Opérationnel

TechNova · Scoring crédit · RAG OpenAgenda · Chess Agent · Triage médical · SortirIA

Docker, Docker Compose

Opérationnel

TechNova · Scoring crédit · RAG OpenAgenda · CheckIt.AI · Chess Agent · Triage médical · SortirIA

Tests et qualité : pytest, ruff, pyright, mypy

Opérationnel

Scoring crédit · CheckIt.AI · Chess Agent · Triage médical · SortirIA

Streamlit

Opérationnel

TechNova · Scoring crédit · RAG OpenAgenda

vLLM

En progression

Triage médical

Angular

En progression

Chess Agent · SortirIA

MLOps & cloud

CI/CD : GitHub Actions — Opérationnel

TechNova · Scoring crédit · RAG OpenAgenda · Chess Agent · Triage médical · SortirIA

MLflow — Opérationnel

Scoring crédit · SortirIA

Weights & Biases — Opérationnel

Triage médical

Airflow — En progression

CheckIt.AI · SortirIA

Monitoring et dérive : Evidently, PSI, Prometheus, Grafana — Opérationnel

Scoring crédit · Triage médical · SortirIA

ONNX Runtime — En progression

Scoring crédit

GPU cloud (RunPod), Hugging Face Spaces — En progression

TechNova · Triage médical

Cloud IA managé : Azure ML, SageMaker, Vertex AI — Notions

À acquérir

DVC, Kubernetes — Notions

À acquérir

Cloud : Azure (Container Apps, PostgreSQL, Static Web Apps) — En progression

SortirIA *Fashion-Insta : architecture conçue*

Conduite de projet

Cadrage et étude de faisabilité — Opérationnel

Fashion-Insta Triage médical

Planning et budget, ROI — En progression

Fashion-Insta SortirIA

Critères de succès, grille go / no-go — Opérationnel

Triage médical SortirIA

RGPD et cadre réglementaire (AI Act) — En progression

Fashion-Insta Triage médical

Langues

Français — Langue maternelle

Anglais — Intermédiaire

Espagnol — Intermédiaire

Ce que demande le marché, et ce que je vais acquérir

Les offres d'AI Engineer en France demandent, en plus de Python et des LLM : Docker, Kubernetes, CI/CD, MLflow et Airflow, un cloud (AWS, GCP ou Azure) et une attention aux coûts et à la latence. Mes projets couvrent déjà la plupart de ces points, et SortirIA tourne sur Azure ; mes priorités sont les suivantes.

1. **Un cloud IA managé** (Azure ML ou AWS SageMaker) : après un service déployé sur Azure Container Apps, entraîner et servir un modèle sur une plateforme dédiée.

2. **Kubernetes** : passer de Docker Compose à un déploiement orchestré.

3. **Versionnement des données** (DVC) et infrastructure en code (Terraform).

4. **Évaluation des LLM en production** : SortirIA suit déjà coûts, versions de consigne, avis et dérive ; reste la qualité mesurée dans la durée, sur un vrai trafic.

CHRONOLOGIE

Mon parcours

EXPÉRIENCE PROFESSIONNELLE

Informatique & électronique

Relation client, compréhension des besoins, conseil et expérience du secteur informatique.

ÉVOLUTION PROFESSIONNELLE

Gestion de projet informatique

Organisation, autonomie et approche orientée résolution de problèmes.

FORMATION INGÉNIEUR IA · OPENCLASSROOMS

Du modèle au système

Octobre 2025 à novembre 2026 : vision, Machine Learning et déploiement, RAG, MLOps, cadrage de projet, apprentissage semi-supervisé, renforcement, ingénierie des données, agents, fine-tuning de LLM, puis SortirIA, mon projet personnel, mis en production sur Azure.

AUJOURD'HUI

AI Engineer

Concevoir des solutions IA complètes, évaluables, reproductibles et déployables.

BILAN

Prendre du recul

Ce que j'ai appris

- La qualité des données prime sur leur quantité.
- Une métrique seule ne suffit pas à juger un modèle : sur le triage, le sous-triage compte autant que le F1.
- L'analyse des erreurs complète l'évaluation chiffrée.
- La reproductibilité doit être pensée dès le début.
- Un juge LLM se calibre sur un humain : chaque version de SortirIA lui a retiré ce que le code vérifie mieux que lui.

Ce que je ferais différemment

- Figer plus tôt les jeux de données.
- Définir les critères de succès avant l'entraînement.
- Automatiser davantage les audits.
- Mieux cibler les expériences réellement informatives.
- Mettre en ligne et surveiller plus tôt, pour mesurer sur de vrais usages.

Les progrès que je constate

- Mes premiers projets s'arrêtaient au notebook ; les derniers sont livrés avec API, tests, CI/CD et documentation.
- Je sais diagnostiquer un échec d'entraînement et le corriger, au lieu de relancer au hasard.
- Je présente mes résultats avec leurs limites.
- Je réutilise mon propre code : SortirIA assemble et adapte des briques de quatre projets précédents.
- Un service en ligne, déployé à chaque push, avec évaluation en CI, tableaux de bord, alertes et budget.

Les défis qui restent

- Tenir une charge réelle, au-delà d'un service à faible trafic.
- Orchestrer avec Kubernetes.
- Mesurer la qualité des réponses d'un LLM dans la durée, avec de vrais utilisateurs.
- Travailler en équipe sur un produit IA déjà en production.

ÉVOLUTION DE MA VISION DU MÉTIER

Au début de la formation

Entraîner un bon modèle

↓

Aujourd'hui

Construire un système IA fiable

Je voyais l'AI Engineer comme quelqu'un qui choisit et entraîne des modèles. Je le vois aujourd'hui comme quelqu'un qui part d'un besoin métier, fixe des critères de réussite, construit toute la chaîne (données, modèle, évaluation, service, monitoring) et aide à décider s'il faut déployer ou non.
La formation m'a surtout appris à mener un projet IA comme en entreprise : cadrer le besoin, chiffrer, mesurer, documenter et rendre compte des limites.

```
Données → expérimentation → modèle → évaluation → déploiement → monitoring → amélioration continue
```

SAVOIR-ÊTRE

Soft skills

Esprit d'analyse

Diagnostic de l'échec du premier entraînement sur Qwen3-Base, mesuré puis corrigé (triage médical).

Curiosité et apprentissage continu

Onze projets dans des domaines différents, du scoring crédit au renforcement, aux LLM et au cloud.

Communication et vulgarisation

Documentation publiée pour chaque projet, rapport technique de 20 pages qui s'ouvre sur une synthèse pour les décideurs.

Gestion de projet et autonomie

POC de triage livré en 4 semaines avec une grille go / no-go ; SortirIA en ligne trois jours après son démarrage, avec une feuille de route jusqu'au gel du code.

Relation client

Des années passées à traduire un besoin client en solution concrète ; pour SortirIA, des partenaires mis en avant et des demandes d'autorisation avant toute collecte.

Créativité

Un agent d'échecs qui combine moteur, RAG et LLM, chacun avec un rôle précis.

CONTACT

Construisons quelque chose d'utile.

Vous souhaitez échanger autour d'un projet IA, d'une opportunité ou simplement parler technologie ?

gonzalezcourrier34@gmail.com

© 2026 Stéphane Gonzalez